

```

import { useState } from "react";

const SECTIONS = [
  { id: "claim", label: "Core Claim" },
  { id: "novelty", label: "Novelty" },
  { id: "pipeline", label: "Pipeline" },
  { id: "spec", label: "Formal Spec" },
  { id: "output", label: "Output" },
  { id: "impl", label: "Implementation" },
];

const pipeline = [
  {
    step: "01",
    title: "Corpus Extraction",
    tag: "DATA",
    color: "#c8a96e",
    desc: "Export raw behavioral history from all platforms – Google Takeout (see",
    artifacts: ["google_history.json", "youtube_history.json", "bookmarks.html",
  },
  {
    step: "02",
    title: "Temporal Node Tagging",
    tag: "PARSE",
    color: "#7eb8c9",
    desc: "Each item is tagged with: domain cluster, concept tokens (NLP-extracte",
    artifacts: ["tagged_nodes.json", "concept_vocab.json"],
  },
  {
    step: "03",
    title: "Traversal Graph Construction",
    tag: "GRAPH",
    color: "#9b8ec4",
    desc: "Nodes are concepts. Edges are co-occurrence within a session window an",
    artifacts: ["traversal_graph.graphml", "edge_weights.json"],
  },
  {
    step: "04",
    title: "Recommendation Delta Extraction",
    tag: "DELTA",
    color: "#c47e7e",
    desc: "The critical step. Recommended-but-unvisited items are extracted as a",
    artifacts: ["frontier_nodes.json", "delta_graph.graphml"],
  },
];

```

```

{
  step: "05",
  title: "Structural Inference",
  tag: "INFER",
  color: "#7ec49b",
  desc: "Run spectral clustering on the combined graph. Identify bridge nodes –
  artifacts: ["clusters.json", "bridge_nodes.json", "gap_analysis.json"],
},
{
  step: "06",
  title: "Ontology Output",
  tag: "OUT",
  color: "#c8a96e",
  desc: "The output is a formal knowledge graph: a candidate ontological skelet
  artifacts: ["domain_ontology.ttl", "gap_report.md", "viz.html"],
},
];

```

```

const noveltyPoints = [
  {
    title: "Existing: Recommendation Systems",
    desc: "Optimize for engagement. The algorithm serves the platform. No one has
    type: "prior",
  },
  {
    title: "Existing: Ontology Learning",
    desc: "Built from text corpus analysis, expert annotation, or co-citation net
    type: "prior",
  },
  {
    title: "Existing: User Modeling",
    desc: "Tracks preferences to serve better ads or content. The user is the pro
    type: "prior",
  },
  {
    title: "EchoGraph: What Is New",
    desc: "Treats the recommendation algorithm as an unintentional domain ontolog
    type: "new",
  },
];

```

```

export default function App() {
  const [active, setActive] = useState("claim");

  return (
    <div style={{
      minHeight: "100vh",

```

```

background: "#0d0d0f",
color: "#e8e2d4",
fontFamily: "'Georgia', 'Times New Roman', serif",
padding: "0",
overflowX: "hidden",
}}>
{/* Header */}
<div style={{
  borderBottom: "1px solid #2a2820",
  padding: "48px 48px 36px",
  background: "linear-gradient(180deg, #0d0d0f 0%, #111109 100%)",
  position: "relative",
  overflow: "hidden",
}}>
  <div style={{
    position: "absolute", top: 0, right: 0, width: "40%", height: "100%",
    background: "radial-gradient(ellipse at 80% 50%, rgba(200,169,110,0.04)
    pointerEvents: "none",
  }} />
  <div style={{ display: "flex", alignItems: "baseline", gap: "16px", margin:
    <span style={{ fontSize: "11px", letterSpacing: "4px", color: "#c8a96e"
      METHODOLOGY · v0.1 · APRIL 2026
    </span>
  </div>
  <h1 style={{
    fontSize: "clamp(2.2rem, 5vw, 3.6rem)",
    fontWeight: "normal",
    margin: "0 0 6px",
    letterSpacing: "-0.02em",
    lineHeight: 1.1,
    color: "#f0ead8",
  }}>
    EchoGraph
  </h1>
  <div style={{ fontSize: "1.05rem", color: "#9a9070", marginBottom: "20px"
    Behavioral Epistemic Reconstruction via Algorithmic Trace Analysis
  </div>
  <p style={{
    maxWidth: "640px", fontSize: "0.95rem", lineHeight: 1.75,
    color: "#b8b09a", margin: "0 0 28px",
  }}>
    A methodology for deriving the latent ontological structure of a knowle
    from years of a researcher's internet traversal history – using the rec
    algorithm as an unintentional domain cartographer.
  </p>
  <div style={{ display: "flex", gap: "10px", flexWrap: "wrap" }}>
    [{"No corpus required", "Behavior-native", "Platform-agnostic", "Zero a

```

```

        <span key={t} style={{
          fontSize: "11px", letterSpacing: "1.5px", padding: "5px 12px",
          border: "1px solid #2e2c24", color: "#7a7260",
          fontFamily: "monospace", textTransform: "uppercase",
        }}>{t}</span>
      )}}
    </div>
  </div>

  { /* Nav */ }
  <div style={{
    display: "flex", gap: "0", borderBottom: "1px solid #1e1c18",
    overflowX: "auto", background: "#0f0f0d",
  }}>
    {SECTIONS.map(s => (
      <button key={s.id} onClick={() => setActive(s.id)} style={{
        padding: "14px 24px",
        background: active === s.id ? "#191812" : "transparent",
        color: active === s.id ? "#c8a96e" : "#5a5648",
        border: "none",
        borderBottom: active === s.id ? "1px solid #c8a96e" : "1px solid tran
        fontSize: "11px", letterSpacing: "2px", cursor: "pointer",
        fontFamily: "monospace", textTransform: "uppercase",
        whiteSpace: "nowrap", transition: "all 0.15s",
      }}>{s.label}</button>
    )
  )}}
  </div>

  { /* Content */ }
  <div style={{ padding: "48px", maxWidth: "820px" }}>

    {active === "claim" && (
      <div>
        <SectionTitle label="The Core Claim" />
        <p style={prose}>
          Recommendation algorithms – YouTube, Google Search, arXiv, Reddit,
          trained on co-occurrence, semantic proximity, and traversal pattern
          of users. After years of sustained exploration within a domain, you
          behavioral history has caused these systems to construct a high-dim
          of how concepts in your field cluster, connect, and sequence.
        </p>
        <p style={prose}>
          That model was built to serve the platform. <strong style={{ color:
          the relationship.</strong> It treats the accumulated recommendation
          specifically as a compressed, empirically grounded encoding of the
          topological structure, derived from your years of intellectual trav
        </p>
      </div>
    )}
  </div>

```

<Callout>

The recommendation trace is not noise. It is a proxy for the struct of the knowledge space you have been navigating – built by an algo no agenda except structural adjacency.

</Callout>

<p style={prose}>

The methodology extracts this signal by computing the delta a researcher has explicitly explored and what the algorithm persist as structurally adjacent but unvisited. That delta is the domain fr set of nodes the algorithm has correctly identified as belonging to knowledge structure but which the researcher's formal framework has incorporated.

</p>

<p style={prose}>

The output is a candidate ontological skeleton of the domain – not corpus analysis, not manually annotated – derived entirely from beh

</p>

</div>

)}

{active === "novelty" && (

<div>

<SectionTitle label="Why This Does Not Exist Yet" />

<p style={{ ...prose, marginBottom: "32px" }}>

The adjacent literature exists. The synthesis does not. Each prior one component of this methodology in isolation – none inverts the relationship to use the algorithm as a research instrument.

</p>

<div style={{ display: "flex", flexDirection: "column", gap: "1px", b {noveltyPoints.map((n, i) => (

<div key={i} style={{

padding: "24px 28px",

background: n.type === "new" ? "#161510" : "#111109",

borderLeft: `3px solid \${n.type === "new" ? "#c8a96e" : "#2a282

<div style={{ display: "flex", alignItems: "center", gap: "12px

<span style={{

fontSize: "10px", letterSpacing: "2px", fontFamily: "monosp

color: n.type === "new" ? "#c8a96e" : "#4a4840",

padding: "3px 8px",

border: `1px solid \${n.type === "new" ? "#c8a96e40" : "#2a2

{n.type === "new" ? "NEW" : "PRIOR ART"}

<span style={{ fontSize: "0.92rem", color: n.type === "new" ?

{n.title}


```

    </div>
    <p style={{ margin: 0, fontSize: "0.88rem", lineHeight: 1.7, co
      {n.desc}
    </p>
  </div>
  )))
</div>
<div style={{ marginTop: "36px" }}>
  <SectionTitle label="The Inversion Claim" small />
  <p style={prose}>
    Every prior use of recommendation systems treats the algorithm as
    provider and the user as a recipient. EchoGraph is the first meth
    that treats the algorithm as a data source and the user's history
    the instrument – reversing the epistemic relationship entirely.
  </p>
  <p style={prose}>
    Critically, this works specifically because recommendation algori
    for structural adjacency in latent concept space. The side effect
    optimization – when applied to a researcher who has spent years i
    domain – is an implicit, continuously updated map of that domain'
    EchoGraph makes this map explicit.
  </p>
</div>
</div>
)}

{active === "pipeline" && (
  <div>
    <SectionTitle label="The Six-Stage Pipeline" />
    <p style={{ ...prose, marginBottom: "40px" }}>
      The methodology is fully implementable with publicly available tool
      Each stage has defined inputs, outputs, and tooling requirements.
    </p>
    <div style={{ display: "flex", flexDirection: "column", gap: "2px" }}
      {pipeline.map((p, i) => (
        <div key={i} style={{
          background: "#111109", padding: "24px 28px",
          borderLeft: `3px solid ${p.color}22`,
          position: "relative",
        }}>
          <div style={{ display: "flex", justifyContent: "space-between",
            <div style={{ display: "flex", alignItems: "baseline", gap: "
              <span style={{ fontFamily: "monospace", fontSize: "11px", c
                {p.step}
              </span>
              <span style={{ fontSize: "1rem", color: "#e8e2d4" }}>{p.tit
            </div>
          </div>
        )
      )
    )
  )
}

```

```

        <span style={{
            fontSize: "9px", letterSpacing: "2px", fontFamily: "monospa
            color: p.color, padding: "2px 8px", border: `1px solid ${p.
        }}>{p.tag}</span>
    </div>
    <p style={{ margin: "0 0 14px", fontSize: "0.87rem", lineHeight
        {p.desc}
    </p>
    <div style={{ display: "flex", gap: "8px", flexWrap: "wrap" }}>
        {p.artifacts.map(a => (
            <code key={a} style={{
                fontSize: "11px", color: "#5a6a50", background: "#0d0d0b"
                padding: "2px 8px", fontFamily: "monospace",
                border: "1px solid #1e1c18",
            }}>{a}</code>
        ))}
    </div>
</div>
    ))}
</div>
</div>
    )}

{active === "spec" && (
    <div>
        <SectionTitle label="Formal Specification" />
        <p style={prose}>The methodology is formally defined over the followi

        <SpecBlock title="Traversal Graph  G_T">
{`G_T = (V_e, E_t, W)

V_e  = { v | v is a concept node derived from explored history }
E_t  = { (u,v) | u,v co-occur within session window  $\tau$  }
W    = edge weight  $\propto$  (co-occurrence frequency  $\times$  recency decay)`}
        </SpecBlock>

        <SpecBlock title="Frontier Set  F">
{`F = V_r \setminus V_e

V_r = { v | v was surfaced by recommendation algorithm }
V_e = { v | v was explicitly visited/consumed }

F   = concepts the algorithm places in your domain
      that your formal work has not yet addressed`}
        </SpecBlock>

        <SpecBlock title="Persistence Score  P(v)">

```

$\{P(v) = \sum_t [\text{rec}(v, t)] / T$

$\text{rec}(v, t) = 1$ if v was recommended at time t , 0 otherwise

T = total time span of history

High $P(v) \in F \rightarrow$ structural gap with high confidence

Low $P(v) \in F \rightarrow$ noise or tangential recommendation`}

</SpecBlock>

<SpecBlock title="Structural Gap Report R">

$\{R = \{ (v, P(v), \text{cluster}(v), \text{bridge_score}(v)) \mid v \in F, P(v) > \theta \}$

θ = persistence threshold (tunable, default 0.15)

$\text{cluster}(v)$ = spectral cluster assignment in $G_T \cup G_F$

bridge_score = betweenness centrality in combined graph

R is sorted by $\text{bridge_score} \times P(v)$ descending.

Top entries are the next research agenda.`}

</SpecBlock>

</div>

)}

{active === "output" && (

<div>

<SectionTitle label="What EchoGraph Produces" />

<p style={prose}>

Three concrete deliverables, each usable independently.

</p>

{[

{

n: "01", title: "Domain Ontology Graph",

file: "domain_ontology.ttl",

color: "#c8a96e",

desc: "A formal RDF/Turtle knowledge graph of the domain as impli

},

{

n: "02", title: "Structural Gap Report",

file: "gap_report.md",

color: "#7eb8c9",

desc: "A ranked list of domain concepts that the algorithm has pe

},

{

n: "03", title: "Temporal Topology Visualization",

file: "viz.html",

color: "#9b8ec4",

desc: "A D3-powered interactive force-directed graph showing the


```

    },
].map(o => (
  <div key={o.n} style={{
    background: "#111109", padding: "24px 28px", marginBottom: "2px",
    borderLeft: `3px solid ${o.color}33`,
  }}>
    <div style={{ display: "flex", justifyContent: "space-between", a
      <div style={{ display: "flex", gap: "14px", alignItems: "baseli
        <span style={{ fontFamily: "monospace", fontSize: "11px", col
        <span style={{ fontSize: "1rem", color: "#e8e2d4" }}>{o.title
      </div>
      <code style={{ fontSize: "11px", color: "#5a5648", fontFamily:
    </div>
    <p style={{ margin: 0, fontSize: "0.87rem", lineHeight: 1.72, col
  </div>
  )})
</div>
)})

{active === "impl" && (
  <div>
    <SectionTitle label="Implementation Entry Point" />
    <p style={prose}>
      The full stack is three Python modules and one config file. No ML t
      All inference runs on a commodity laptop.
    </p>

    <CodeBlock title="install">{`pip install networkx sentence-transformer
rdflib google-takeout-parser`}</CodeBlock>

    <CodeBlock title="echograph/extract.py">{`from google_takeout_parser

def build_node_set(takeout_path: str) -> list[dict]:
    """
    Returns tagged nodes from Google Takeout export.
    Each node: { url, title, timestamp, domain, concepts[] }
    """
    records = parse_takeout(takeout_path)
    return [tag_concepts(r) for r in records]`}</CodeBlock>

    <CodeBlock title="echograph/graph.py">{`import networkx as nx

def build_traversal_graph(nodes, session_window=1800) -> nx.Graph:
    """
    session_window: seconds. Nodes co-visited within
    this window share an edge. Weight = frequency.
    """

```

```

G = nx.Graph()
for i, n in enumerate(nodes):
    G.add_node(n['url'], **n)
    for m in nodes[i+1:]:
        if abs(n['ts'] - m['ts']) < session_window:
            if G.has_edge(n['url'], m['url']):
                G[n['url']][m['url']]['weight'] += 1
            else:
                G.add_edge(n['url'], m['url'], weight=1)
return G`</CodeBlock>

```

```

<CodeBlock title="echograph/frontier.py">{`def compute_gap_report(G_e
"""
Returns ranked structural gaps:
{ node, persistence, cluster, bridge_score }
"""

frontier = set(G_recommended.nodes) - set(G_explored.nodes)
combined = nx.compose(G_explored, G_recommended)
centrality = nx.betweenness_centrality(combined)

report = []
for v in frontier:
    p = persistence_score(v, G_recommended)
    if p > theta:
        report.append({
            'node': v,
            'persistence': p,
            'bridge_score': centrality.get(v, 0),
        })

return sorted(report, key=lambda x: x['persistence'] * x['bridge_score'],
               reverse=True)`</CodeBlock>

```

```

<div style={{ marginTop: "32px" }}>
    <Callout>
        The repository structure is: <code style={{ fontFamily: "monospace"
        CLI entry: <code style={{ fontFamily: "monospace", fontSize: "0.8
    </Callout>
</div>
</div>
)}

```

</div>

{/* Footer */}

```

<div style={{
    borderTop: "1px solid #1a1814", padding: "24px 48px",

```

```

        display: "flex", justifyContent: "space-between", alignItems: "center",
        flexWrap: "wrap", gap: "12px",
    }}>
    <span style={{ fontFamily: "monospace", fontSize: "11px", color: "#3a3828"
        ECHOGRAPH · BEHAVIORAL EPISTEMIC RECONSTRUCTION · APRIL 2026
    </span>
    <span style={{ fontFamily: "monospace", fontSize: "11px", color: "#3a3828"
        quantummorph.com / github.com/Anurag1
    </span>
    </div>
</div>
);
}

// — Shared Components —————

function SectionTitle({ label, small }) {
    return (
        <div style={{ marginBottom: "28px" }}>
            <div style={{
                width: "24px", height: "1px", background: "#c8a96e",
                marginBottom: "12px",
            }} />
            <h2 style={{
                margin: 0,
                fontSize: small ? "0.9rem" : "1.1rem",
                fontWeight: "normal",
                color: "#e8e2d4",
                letterSpacing: "0.01em",
            }}>{label}</h2>
        </div>
    );
}

function Callout({ children }) {
    return (
        <div style={{
            margin: "28px 0",
            padding: "20px 24px",
            background: "#13110e",
            borderLeft: "2px solid #c8a96e",
        }}>
            <p style={{ margin: 0, fontSize: "0.93rem", lineHeight: 1.75, color: "#b0a8"
                {children}
            </p>
        </div>
    );
}

```

```
}
```

```
function SpecBlock({ title, children }) {
  return (
    <div style={{ marginBottom: "24px" }}>
      <div style={{
        fontSize: "10px", letterSpacing: "2px", fontFamily: "monospace",
        color: "#5a5040", marginBottom: "8px",
      }}>{title}</div>
      <pre style={{
        background: "#0c0c0a",
        border: "1px solid #1e1c18",
        padding: "20px 24px",
        fontFamily: "monospace",
        fontSize: "12px",
        color: "#8a9e7a",
        lineHeight: 1.8,
        margin: 0,
        overflowX: "auto",
        whiteSpace: "pre",
      }}>{children}</pre>
    </div>
  );
}
```

```
function CodeBlock({ title, children }) {
  return (
    <div style={{ marginBottom: "20px" }}>
      <div style={{
        fontSize: "10px", letterSpacing: "2px", fontFamily: "monospace",
        color: "#4a4840", marginBottom: "6px", display: "flex",
        justifyContent: "space-between",
      }}>
        <span>{title}</span>
        <span style={{ color: "#2e2c24" }}>>python</span>
      </div>
      <pre style={{
        background: "#0a0a08",
        border: "1px solid #1a1814",
        padding: "18px 22px",
        fontFamily: "monospace",
        fontSize: "12px",
        color: "#7a9a6a",
        lineHeight: 1.8,
        margin: 0,
        overflowX: "auto",
        whiteSpace: "pre",
      }}>
```

```
    }}>{children}</pre>
  </div>
);
}
```

```
const prose = {
  fontSize: "0.93rem",
  lineHeight: 1.82,
  color: "#8a8270",
  marginBottom: "20px",
  maxWidth: "660px",
};
```